University of Waterloo
Faculty of Engineering
Department of Electrical and Computer Engineering

Detailed Design

Podcasts Application with Automatic Advertisement Skipping and Recommendation Engine

Group 2017.001

Prepared by

[redacted]

Consultant: Prof. Stephen Smith

June 30, 2016

This report earned an overall score of 65.6 / 80.

Table of Contents

1 High Level Description

1.1 Motivation

There were 60 000 active podcasts on the US iTunes store in 2015, which is a massive increase from around 10 000 podcasts in 2008 [1]. Meanwhile, the percentage of Americans consuming podcasts increased from 9% to 17% in the same time frame [2]. This data points to a peculiar scenario where the podcast market is growing faster in supply than it is in demand. One of the reasons for this situation could be that existing podcast clients use outdated technologies that are not optimized for mobile devices and that do not provide their users with interesting services based the data they collect from them. LibSyn, one of the largest podcast hosting companies, reports that of the 2.6 billion podcasts downloaded from them in 2014, 63% were requested from mobile devices which is up from 43% in 2012 [2]. With the increasing prevalence of mobile devices and the stagnant feature set of current offerings, podcast clients are ripe for technical advancement and improvement.

1.2 Project Objective

The objective of this project is to design a podcast client called ByteCast that makes consumption more appealing to users by incorporating features such as advertisement-skipping, smart recommendations and better shareability of podcasts on social media. Current podcast clients are lacking in these types of features which digital-age consumers have grown to expect from media providers such as Netflix.

1.3 Block Diagram

The proposed solution is a podcast client that has a dedicated backend service capable of generating recommendations and inferring advertisement locations based on crowdsourced user input. The architecture is shown in **Figure 1** and is composed of four main subsystems: The iOS (iPhone operating system) application subsystem, the web application subsystem, the backend server subsystem and the recommendations subsystem.

1.3.1 iOS Application Subsystem

The iOS Application is broken down into five major components that are logically separate. The first component is the networking layer which facilitates the communication between the client itself, the backend server subsystem and Apple iTunes podcast database. The data received from the networking layer is managed through the controller subsystem. Depending on the data being supplied, the controller persists the data locally through the persistent storage component, provides a data source to the audio playback component or provides information for the user interface. The persistent storage component manages fetching and storing audio files, the associated metadata, user interaction statistics as well as user information. The audio playback component outputs the podcast audio to the user when supplied a data stream from the controller component. The user interface takes inputs from the user and outputs the corresponding visual response.

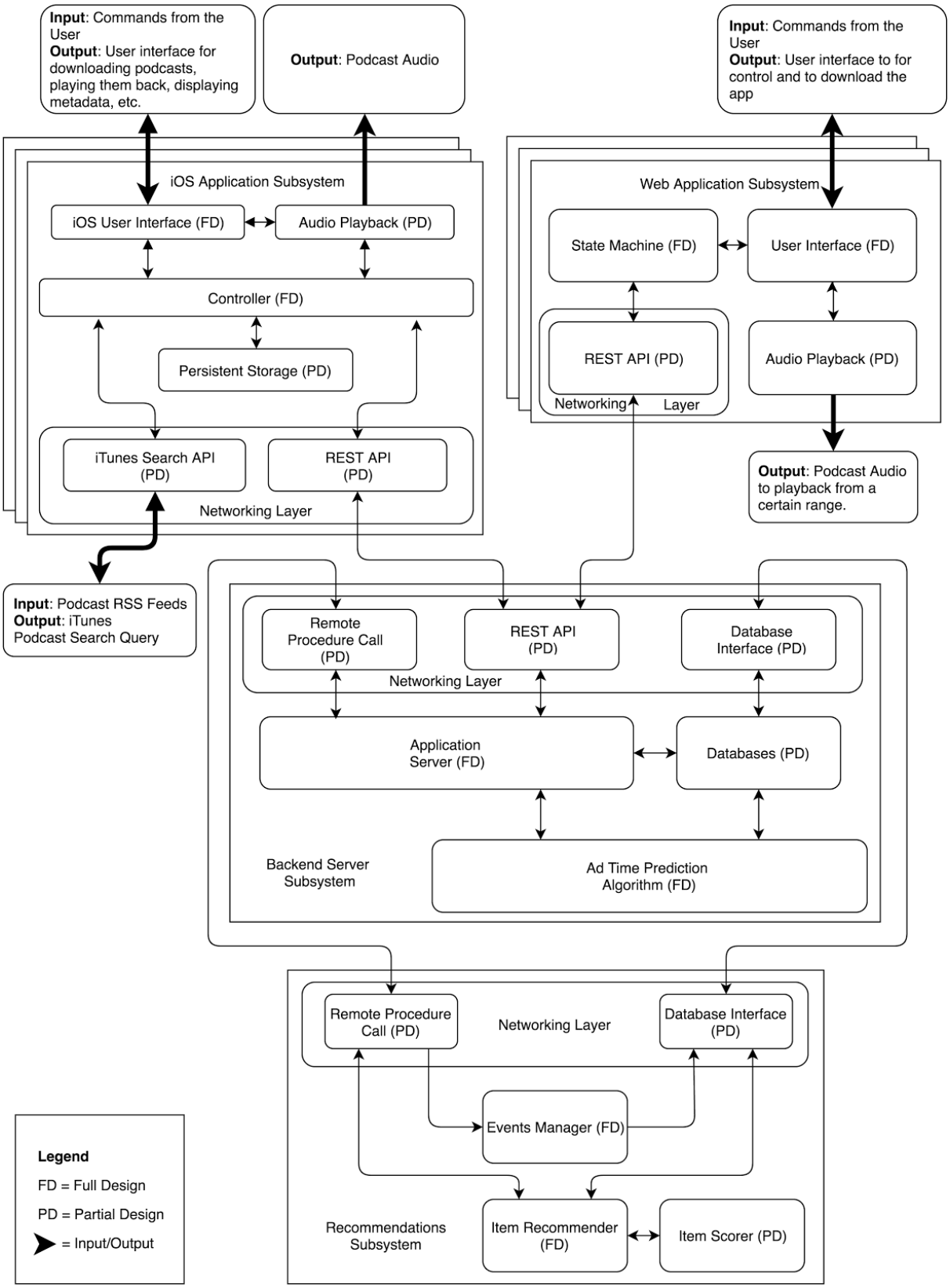


Figure 1: A block diagram showing the software components of the application

1.3.2 Web Application Subsystem

The web application contains four components that are architecturally separated by their use case. The first subsystem is the network layer which communicates with the backend server through the use of a REST API. REST APIs are application program interfaces that expose a set of functionality useful for building out client frontends that interact with web service backends [3]. Since the main purpose of the web application is to play back a certain range of a podcast in the user's web browser, the data being passed contains a feed to the audio file in question and a timestamp for the playback period. The state machine has to save this information while the session is open. Finally the user interface allows the user to view the metadata associated with the audio file being sent and play/stop the audio. The audio playback portion of the web application mirrors that of the iOS and allows the user to navigate the audio file. Lastly the web app also includes an embedded link in the user interface to download the iOS app.

1.3.3 Backend Server Subsystem

The backend server contains four major components that are logically separate. All communication with external systems is handled by the networking layer which consists of three sub-components: the Remote Procedure Call (RPC) interface, the REST API and the Database interface. A Remote Procedure Call (RPC) is a technique for making a function call from one software system to another over a network [4]. The Application Server holds the business logic of the RPC and API calls and directly interacts with the database to manage data. The Database Interface is used exclusively for external access to the database from the recommendations subsystem. The last component is the Ad Time Prediction Algorithm which is an algorithm used by the Application Server to infer the advertisement times of a given podcast episode based on collected user data that is present in the database.

1.3.4 Recommendations Subsystem

The recommendations system contains four different components that are logically separate. Almost all interactions with the recommendations system must go through the networking layer, which is itself composed of the RPC sub-component and the Database Interface sub-component. The Events Manager component is an important part that serializes, categorizes and stores the events to persistent storage such as a database. Due to the fact that the events need to be stored in the database, this part is very closely coupled with the Database Interfacing sub-component. Lastly, the Item Recommender and Item Scorer are two related components that also work together. The Item Recommender packages, sorts and caches recommendation results. The Item Scorer is responsible for computing the recommendation score for every podcast with relation to the user and returns the top recommended podcasts using the data provided by the overall subsystem [5].

2 Project Specifications

The project specifications can be divided into functional specifications and non-functional specifications. Each specification is categorized as either essential or nonessential based on its importance.

2.1 Functional Specifications

Ten functional specifications were identified and can be found in **Table 1**.

Table 1: Functional Specifications

Subsystem	Specification	Additional Details	Necessity
iOS Application	iOS application must be able to automatically skip over portions of the podcast that the server component has deemed to be advertisements.	In order to verify that this subsystem is working properly, automated test cases are added to this block. These tests are able to verify that the application does not play back portions of the podcast that are marked as advertisements.	Essential
iOS Application	iOS application must be able to playback locally downloaded and streaming podcasts. When streaming, there must be no buffering on networks with at least 1 Mbps of downlink capacity.	These are standard playback features that are present in competitive products. To verify the streaming performance, the network degradation utilities built into the iOS Simulator can be used.	Essential
iOS Application	Statistics that iOS Application must track: The time ranges of the podcast that the user skipped over and the timestamp at which the user stops listening to the podcast entirely.	This data must be collected on the iOS application for the Ad Time Prediction Algorithm component to work.	Essential
iOS Application	The iOS application should be able to simultaneously download or stream at least 2 different MP3 files from a remote URL while on Wi-Fi.	The app allows the user to simultaneously download multiple podcasts to listen to offline.	Non-Essential

Backend Server	Backend Ad Time Prediction Algorithm must interpret user actions given to the server to determine the advertisement ranges within the podcast, accurate to within ±5 seconds for podcasts that have inferrable data from at least 10 different users.	Inferrable data includes skipping forward / fast-forwarding when an advertisement starts playing, explicitly marking advertisement ranges using the user interface, or answering questions about whether or not they just heard an advertisement.	Essential
Backend Server	The user database must be able to hold up to 5000 user's information. The podcast database must be able to hold podcast metadata for up to 10 000 podcasts.	This must account for the ad skipping data, user podcast recommendations, and collected metadata on all podcasts.	Essential
Backend Server / Web Application	The backend must be capable of generating a URL link which can be shared with various social media platforms such as Facebook Messenger or Slack. The URL must be live for at least a week and must direct the user to an interface where they can listen to a small clip of a podcast.	This allows users to listen to small clips of podcasts without having the mobile application. They can also visit the site to listen to podcasts and to download the app.	Non-Essential
Recommendations	Must provide at least 3 recommendations to a specific user based on existing user data.	The recommendations are returned in an unordered set to not impose artificial ranking.	Essential
Recommendations	The Recommendations Subsystem must be able to generate recommendations in under 25 seconds per user. This is the amount of time it takes to generate a model and compute recommendations for a user.	This is permissible because the recommendations are not generated on demand but instead are computed and refreshed on a regular schedule.	Essential
Recommendations	The system must generate recommendations for every user once a day if the user data has changed.	The user data would include any new podcasts a user has listened to or subscribed to.	Non-Essential

2.2 Non-functional Specifications

Three non-functional specifications were identified and can be found in **Table 2**.

Table 2: Non-functional Specifications

Subsystem	Specification	Additional Details	Necessity
Overall System	The system should have strong computer security.	User authentication should happen in a simple and secure manner. Users should only be able to access data that they have proper authorization to view. The backend infrastructure and database tables should only be accessible by the product engineers.	Essential
Backend Server	System should be resilient to malicious behaviour. Users should not be able to spam the servers with requests.	Unauthorized use of the API could cause the system to go down due to overloading the server.	Non-Essential
iOS Application	The application should have battery and data usage that is competitive with the native iOS Podcasts application. It should perform as well as existing podcast clients and should not to be a burden on the hardware.	The app collects a large amount of data in order to better serve the user. While the iPhone is unplugged, the application should use at most 20% more power than the native iOS Podcasts application. The application waits until the phone is charging and on WiFi to do computationally expensive tasks.	Non-Essential

3 Detailed Design

3.1 iOS Application Subsystem Design

3.1.1 Networking Layer Design

The networking component communicates with two different services: the iTunes Search API and the Backend Server Subsystem. Requests for fetching or posting data are made using HTTP requests. The iTunes Search API is used to fetch the metadata associated with a podcast as well as the podcasts corresponding RSS Feed. The RSS feed contains the information necessary for a podcast client to subscribe to the podcast and download new episodes. Podcast metadata is sent to the backend server subsystem whenever a user chooses to subscribe to a podcast. The iOS subsystem uses Alamofire, which is a third party library that creates an abstraction layer to simplify networking on iOS applications.

The networking layer also contains an audio downloader class which asynchronously downloads an audio file given a URL for the file. It is able to handle multiple downloads by dispatching each download onto its own thread. The following pseudocode in **Figure 2** shows how each download is distributed across a different thread.

```
let queue = dispatch_queue_create(urlString, nil)
dispatch_async(queue) {
   Alamofire.request(.GET, self.urlString)
      .stream { (data) in
         self.data.appendData(data)
         dispatch_async(dispatch_get_main_queue()){
            self.delegate?.audioStreamer?(self, dataDidStream: data)
         }
      }.response { _ in
         dispatch_async(dispatch_get_main_queue()) {
            self.delegate?.audioStreamer?(self, didFinishDownloadingData: NSData(data: self.data))
         }
      }
}
```

Figure 2: Pseudocode for asynchronous download requests

The class also has an implementation in place to allow for real-time updates on the progress of the download by using the iOS delegation pattern. The delegation pattern is a commonly used design pattern for message passing between different objects [6]. The callback "*dataDidStream:*" is called periodically and passes the raw data that has been downloaded. There is also a callback called "*didFinishDownloadingData:*" that is called when the download is complete. This design allows for as many downloads as there are available threads in the iOS thread pool. The functional specification for the iOS application being able to simultaneously download or stream at least two different MP3 files while on Wi-Fi without buffering on a 1 Mbps downlink is achieved with this design.

3.1.2 Persistent Storage Layer Design

The iOS application requires a persistent storage architecture for the management of podcast metadata, user statistics and podcast audio files. User statistics are persisted so the application does not spam the backend server subsystem with statistics whenever a viable event occurs. The main event that is tracked is when a user skips forward or backward 15 seconds in the podcast. These are stored in a database and then an HTTP request is made periodically in order to update the backend server subsystem. A periodic request schedule allows the requests to be efficiently batched, resulting in an application that does not perform unnecessary requests. This creates a subsystem that is extremely battery and data conscious.

There are three possibilities for the persistent storage management architecture for the metadata and user statistics: Core Data, SQLite using FMDB and Realm.io. In order to determine the best possible solution, the following criteria must be evaluated: (A) defining models and using them in code and performance metrics, (B) compile time, (C) the number of queries per second and (D) the number of insertions per second.

3.1.2.1 Defining Models and Using them in Code

Core Data is the simplest of the three to implement in code since it is a stock library provided by Apple, meaning that no external libraries or dependencies are required. Apple also provides a simple graphical user interface (GUI) for adding the models into code, which makes creating model objects and relations between them simple and straightforward. One issue is that Core Data requires class files to be regenerated into a difficult to read Objective-C header file each time there is a change to the model object.

FMDB is one of the most widely used third party SQLite wrappers[7] for iOS applications, and is the approach most commonly used when implementing a SQLite database within an existing iOS application. FMDB only acts as a wrapper to simplify interactions to a SQLite database. Each model needs to have some sort of implementation translating it into a valid SQL database query to update the appropriate row(s).

Realm.io is a new open source project that claims to be the ideal cross platform solution for integrating data persistence into mobile applications [8]. Realm is a more comprehensive persistent storage solution that provides a base "*Object*" class for creating models that can be stored into the database. The following pseudocode in **Figure 3** shows the implementation of a podcast metadata model object in Realm:

```
class Metadata: Object {
   dynamic var title: String = ""
   dynamic var id: String = ""
   dynamic var album: String?
   dynamic var artist: String?
   dynamic var albumArtwork: UIImage?
}
```

Figure 3: Podcast Metadata class definition

3.1.2.2 Performance Metrics

For the compile time comparison, a blank iOS application is used in order to keep all other variables constant when testing. XCode provides a method for measuring the compile time for a project, which is used to make the measurements shown in **Figure 4**. Ten measurements are made for each of the different options and the average compile time is computed, which ensures an accurate compile time result for each of the different frameworks. The results show that Realm has the largest compile time difference between all three options by a very large margin, making it the worst choice in this aspect.

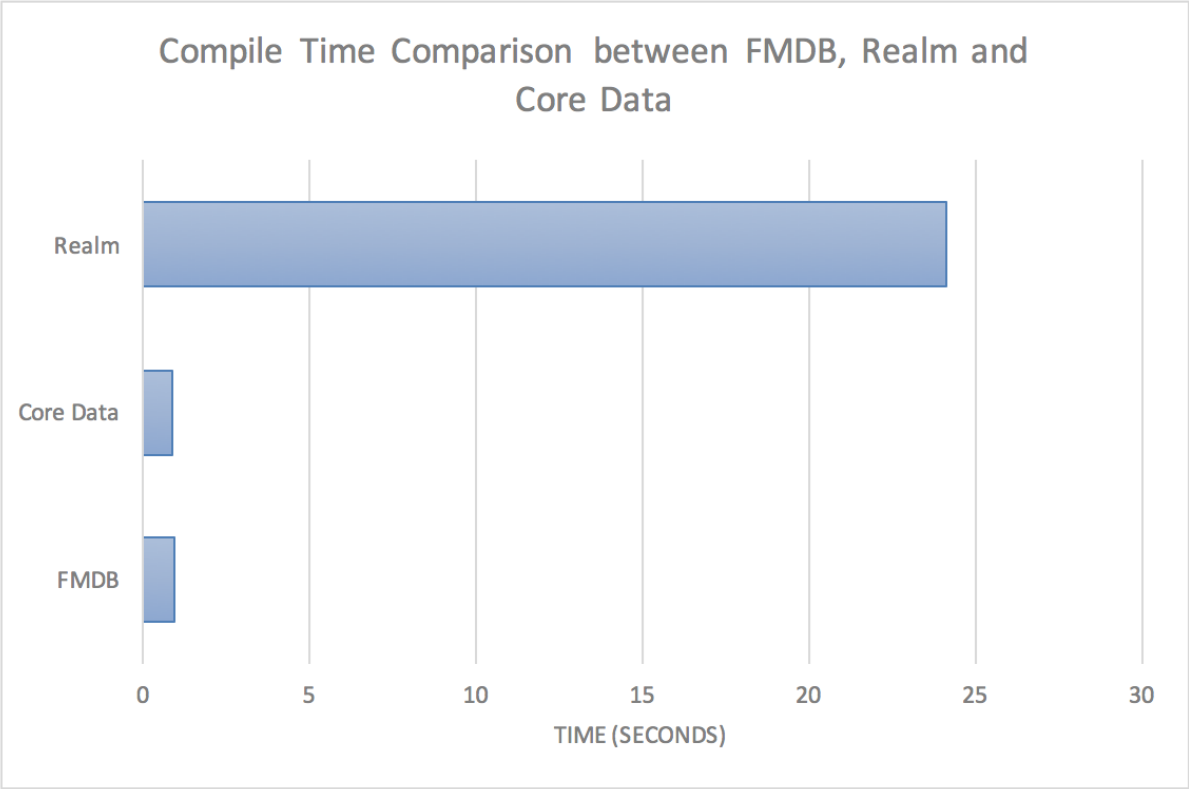


Figure 4: Compile time comparison between FMDB, Realm and Core Data

Realm has provided performance metrics for common queries and made in depth comparisons against other persistent storage options that are available to iOS developers. The test first performs a count on an existing database and the second inserts 200000 entities into an empty database. This helps ensure that the bottleneck of each storage solution is observed. The first query, shown in in **Figure 5**, performs a COUNT on the database and measures the number of queries per second. The second query, shown in **Figure 6**, measures the number of records inserted per second. Realm outperforms Core Data and FMDB in the first performance test while FMDB is superior when making insertions.

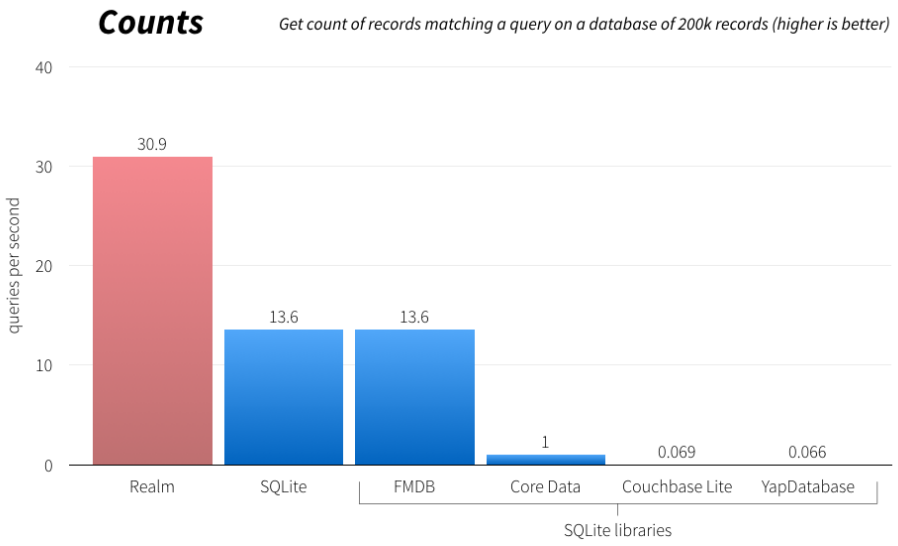


Figure 5: Get a count of records on a 200 000 record database [9]

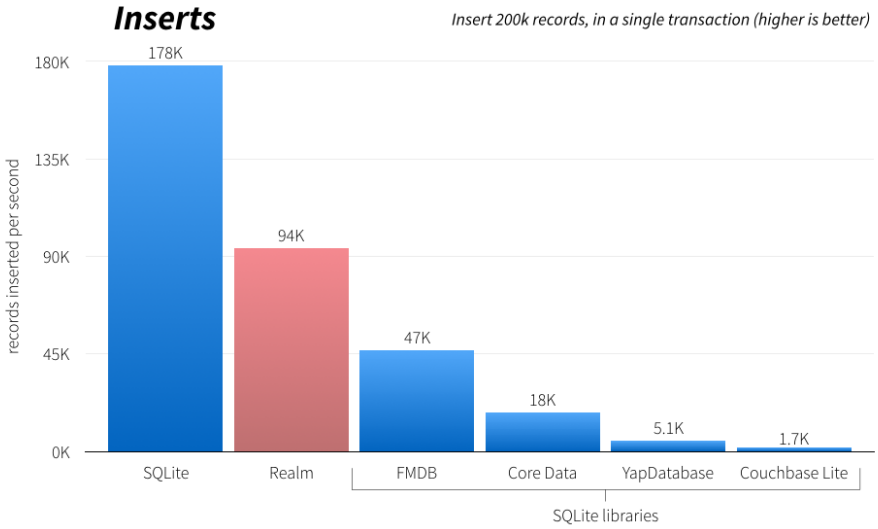


Figure 6: Inserting 200 000 records for each database framework [9]

Table 3 uses the following criteria to determine the best persistent storage solution: (A) defining models and using them in code and performance metrics, (B) compile time, (C) the number of queries per second and (D) the number of insertions per second. Based on the results of the decision matrix, it is apparent that Realm is the best solution between all of the different persistent storage frameworks.

Table 3: Decision Matrix for choosing a persistent storage framework

	A (20%)	B (20%)	C (30%)	D (30%)	Score
Core Data	0.6	0.9	0.4	0.2	0.48
SQLite	0.3	0.9	0.3	0.9	0.60
Realm.io	0.9	0.2	0.8	0.6	0.64

3.1.3 Audio Player Design

Apple provides a low level API called Audio Toolbox [10] which provides an interface for recording, playback and stream parsing of audio. It is written in C and provides a direct interface with the iPhone audio hardware. Apple also provides another API called Core Audio [11], which is an abstraction layer built on top of Audio Toolbox that provides a simpler playback interface.

The iOS Subsystem uses a third party library called AudioPlayer [12], which is a light wrapper on top of Core Audio and provides a basic interface for streaming, playing and pausing local and remote audio files. It provides connection handling, audio queueing as well as performing quality control based on current network conditions. Given the ranges of audio to skip over, AudioPlayer is able to skip over portions of the podcast that are deemed by the backend subsystem to be advertisements. AudioPlayer provides the base functionality of audio playback and is highly customizable, making it an ideal starting point for building an audio player that suits the needs of playing back podcasts.

The player design also incorporates the singleton pattern to manage a global play state since there can only be one audio file playing at any given time. A singleton is generally defined as a class that "returns the same instance no matter how many times an application requests it." [13] A class called "*Player*", shown in **Figure 7**, is initialized as a singleton and it has a member variable that is the third party AudioPlayer class:

```
class Player: NSObject,  AudioPlayerDelegate {
   static let sharedInstance = Player()
   private var audioPlayer: AudioPlayer = AudioPlayer()
   ...
}
```

Figure 7: Class definition of a Player Singleton

3.1.4 User Interface Design

The User Interface (UI) for the iOS Subsystem is built using the fundamental UI library provided by Apple called UIKit [14], which provides a massive set of base classes for building iOS applications.

The navigation scheme of the application is built on top of a UITabBarController [15], which provides a set of tabs at the bottom of the app. This allows the user to navigate between different view controllers

that display podcast recommendations, the user's library and the user's personal settings. The view controllers are organized in a very modular and extensible format. Any view controller that forms the basis of the root navigation of the application simply needs to conform to a custom protocol. Code for the protocol definition in **Figure 8** is as follows:

```
protocol Viewable {
   var displayTitle: String { get }
}
```

Figure 8: Protocol definition for a Tab Bar Controller

For a view controller to conform to this protocol, it needs to implement the protocol as shown in **Figure 9**.

```
class UnplayedViewController: UIViewController, Viewable {
   var displayTitle: String {
      return "Unplayed"
   }
}
```

Figure 9: Class definition of a UIViewController conforming to the Viewable property

The tab bar implementation expects a view controller that conforms to this protocol and uses the variable "displayTitle" to generate the titles of the view controllers dynamically.

3.1.5 Controller Design

The controller in the iOS subsystem acts as the connection between the user interface components and the network layer and persistent storage layer. It contains the model objects as well as a JSON parser that parses the response from the backend server subsystem as well as the iTunes Search API.

In order to create an extensible way for a JSON response to be parsed into its corresponding model object, a protocol called "JsonParser" is used to manage all parseable objects. The following code in **Figure 10** snippet is the definition of the JsonParser protocol:

```
protocol JsonParser {
   associatedtype T
   func parse(dict: [String: AnyObject]) -> T?
}
```

Figure 10: Protocol definition for a JSON Parser

"*Associatedtype*" [16] represents a generic templated type in Swift and can be set as any class type. Every Model object has its own parser that conforms to the JsonParser protocol. This allows for a generic network request architecture that simply needs to expect an object parser that conforms to

JsonParser. The following code snippet in **Figure 11** is the definition of a generic podcast object and its corresponding parser. Please note that the implementation of how a JSON response is parsed has been left out for simplicity.

```
class Podcast: NSObject {
   let title: String
   let id: String
   let feedUrl: String
   let author: String
   var episodeCount: Int
   var artwork: [String: String] = [:]
   var releaseDate: String?
}

class PodcastParser: JsonParser {
   typealias T = Podcast

   func parse(dict: [String : AnyObject]) -> T? {...}
}
```

Figure 11: Class definition for a Podcast object and its corresponding parser

This allows the User Interface, Networking Layer and Persistent Storage layer to use a simple model class for all the tasks that each component is required to do. This ties every single component in the iOS subsystem together into a unified application.

3.2 Web Application Subsystem Design

The web application component exists to serve two main purposes: to provide a mechanism for audio sharing and to allow for data visualization. The web application is the primary way users share snippets of a podcast from the iOS app. The shared audio can then be embedded into other communication platforms such as Facebook Messenger or Slack through the use of HTML. Data visualization is handled through the D3.js charting library and displays the data collected in the iOS application. The web app is mainly written in HTML, Javascript, and CSS.

The web app runs using Webpack on an Amazon AWS EC2 instance. It communicates to the backend server using asynchronous HTTP calls with the Promise.js framework. These asynchronous calls communicate with RESTful endpoints on the backend, which return JSON objects to the web app.

Choosing a framework for the web application is an important decision that influences the performance and extensibility of the application. The main framework choices are React/Redux, AngularJS 1.x, Angular 2, Ember.js, or Polymer. The two most important metrics are performance and extensibility. Performance is important because audio snippets may be accessed through mobile devices which are not as performant as personal computers or laptops. Extensibility is important because each framework uses different design patterns and architectures which makes it difficult to interchange them. Speed tests are performed through the use of MonsterDB to obtain the render speed (page repaint rate per

second). Page repaint rate is a metric used to measure web application performance that is similar to the refresh rate metric used in video games and animations. The main performance criteria include the size of the framework, which affects load times, and the render speed. The main extensibility criteria include the licensing type and project status. **Table 4** shows the results of testing and research on the variety of parameters that leads to our final decision. In **Table 5**, each category is weighed to find the strongest candidate. **Table 5** shows that Ember.js is the best framework to use according to the chosen weighting.

Table 4. Metrics for measuring the five frameworks

	Creators use own framework	Github contributors	Render speed (Paints per second)	Stars on Github	Members of core team
AngularJS 1.x	No	1,479	52.5	49k	N/A
Angular 2	No	279	85	12k	20+
Ember	Yes	589	62.5	16k	12+
Polymer	Partial	84	55	15k	20+
React + Redux	Initially	716	47.5	43k	7+

	License	Size (kb)	Open issues on Github	Status	Communication
AngularJS 1.x	MIT	158	744	Deprecated	Quarterly+
Angular 2	MIT	698	1,354	Alpha	Quarterly+
Ember	MIT	435	176	Stable	Weekly+
Polymer	BSD	222	427	Stable	Monthly
React + Redux	BSD*	157	483	Stable	Monthly

Table 5. Decision matrix based on metrics for the five frameworks

	Creators use own framework (7.5%)	Github contributors (2.5%)	Render speed (Paints per second) (20%)	Stars on Github (2.5%)	Members of core team (2.5%)
AngularJS 1.x	0.000	0.025	0.124	0.025	0.000
Angular 2	0.000	0.005	0.200	0.006	0.025
Ember	0.075	0.010	0.147	0.008	0.015
Polymer	0.038	0.001	0.129	0.008	0.025
React + Redux	0.038	0.012	0.112	0.022	0.009

	License (30%)	Size (kb) (20%)	Open issues on Github (10%)	Status (2.5%)	Communication (2.5%)	Total
AngularJS 1.x	0.300	0.199	0.024	0.000	0.002	0.697
Angular 2	0.300	0.045	0.013	0.008	0.002	0.604
Ember	0.300	0.072	0.100	0.025	0.025	0.777
Polymer	0.300	0.141	0.041	0.025	0.006	0.715
React + Redux	0.100	0.200	0.036	0.025	0.006	0.560

Some of the heavily weighted items include the license used and framework size which are important aspects of extensibility, as well as paints per second which is an important performance metric. React has a patent clause under its copyright which gives Facebook (the creators of React) additional protection against companies that use the framework. This makes React a poor choice in terms of extensibility. The audio sharing component of the application is just a single HTML element therefore framework size has a large effect on its loading time.

3.3 Backend Server Subsystem Design

The backend server component is the core of the ByteCast application and thus has many associated design decisions. Decisions revolve around which hosting provider to make use of, what technologies to use for the server and database, how to design the database schema, what the API service should look like, and the design of the ad time prediction algorithm.

3.3.1 Hosting Provider

Choosing a hosting provider is a crucial first step since it directly affects which technologies can be used for the software stack and thus directly influences any further decisions related to the development of the application. For this project, using home servers is not an option due to limited home bandwidth and poor reliability of the hardware. As a result, only commercial hosting options are considered. The available choices for hosting providers are Amazon Web Services (AWS), Heroku and Google App Engine. The major priorities are to minimize cost, maximize flexibility and a minor emphasis on having easy scalability.

Hosting providers can be divided into two major sections: Infrastructure as a Service (IaaS) and Platform as a Service (PaaS). AWS is an IaaS meaning it provides general purpose machines that can be configured to the application while Heroku and App Engine are grouped into PaaS meaning they handle the operation of the servers and automatically scale and perform load balancing. There are advantages and disadvantages to each approach. IaaS systems allow for more customizability since they give the developer direct access to a server and allow them to choose all aspects of the setup such as the operating system to use, the language to write their applications in, and the database to use. PaaS systems are more restrictive in that they only support a small number of development languages and often require the developer to use a either a proprietary database or choose from a small number of database options. The advantage of PaaS systems is that they require much less configuration and automatically handle a lot of scalability issues by allocating machines on-the-fly in order to keep up with demand.

In terms of price, the main priority is to use a service that has a free-tier in order to avoid racking up a large non-refundable bill for the project. All three services offer free trials of some sort, but they vary greatly in their usefulness. AWS offers a fairly generous 1-year free trial that includes features such as access to a single server with 1 GB of ram and 30 GB of storage, 15 GB per month of outbound bandwidth and 20 GB of managed database storage with automatic scaling using PostgreSQL, MySQL, or MariaDB, and 25 GB of Amazon DynamoDB database storage [17]. Since AWS is an IaaS that provides the developer with high levels of control, it is also possible to network together several free trial

machines in order to separate out components onto different machines. Heroku also offers a free tier [18], but the service is more restrictive in that the server sleeps after 30 minutes of inactivity, the provided server only has 512 MB of ram, and the PostgreSQL database is limited to 10000 rows [19]. These limitations make Heroku's free tier unappealing from a performance and usability standpoint. Finally, Google App Engine provides no free tier and instead provides $300 in credits that can be used over the course of 60 days. Once the 60-day period is over, all instance runtimes, network traffic, and database storage is charged based on usage [20]. Using this model, a small mistake in the code can result in a very large bill that the team would need to pay.

AWS is by-far the most flexible out of the three options since it allows allow aspects of the service to be customized and supports a wide variety of technologies. Since it provides the developer with a server and storage, it is possible to use any language, framework and database that is supported on Linux. To take advantage of the 20 GB of free managed storage, the developer has the choice of using PostgreSQL, MySQL, or MariaDB. Migrating away from the platform and onto your own hardware is also straightforward since everything is installed on a standard Linux distro. Heroku almost exclusively uses PostgreSQL for database storage, but still supports a fairly wide variety of server technologies such as Node.js, Ruby, Java, and Python. Finally App Engine supports Python, Java, PHP and Go languages for development and requires use of custom Google databases such as Datastore, BigQuery, and Cloud SQL which is a managed MySQL database.

Finally, there is minor priority in using a service that is scalable without any additional setup. Since the project requirements are for a low number of users, this criterion is not heavily weighted. Heroku and App Engine offer easier out-of-the-box server scalability, with AWS requiring more manual configuration. All three services offer roughly equivalent solutions for scalable databases, but AWS and App Engine offer a better variety. **Table 6** compares the alternatives:

Table 6: Decision matrix for hosting providers

Alternative	Cost (50%)	Flexibility (30%)	Scalability (20%)	Total
AWS	0.9	0.8	0.6	0.81
Heroku	0.7	0.6	0.9	0.71
App Engine	0.5	0.5	0.9	0.58

After considering each alternative in all the criteria, AWS scores the highest due to its low cost and high flexibility. To secure the AWS server against abuse, port blocking is being used so that all ports other than SSH and HTTP are blocked by default. IP whitelisting is also being employed to selectively open up additional ports to home IP addresses when necessary.

3.3.3 Database Design

An appropriate database and schema is required in order for key features such as automatic ad skipping and podcast recommendations to be effective. Most group members are already familiar with PostgreSQL, and along with its support for JSON data types, it suits the role of database for this project. Using PostgreSQL also allowed for use of the free 20 GB AWS managed storage, which is more than enough room to handle the storage of 5000 user accounts and metadata for 10 000 podcasts. The schema is the more important aspect of the database as it needs to be designed to hold metadata for podcasts and their respective episodes in addition to user data in a simple yet intuitive way.

There are two ways the podcast and episode data could be structured:

1. Have one table for podcasts channels and another table of episodes where each episode has a foreign key to its respective podcast. This schema has the benefit of being extremely simple with the downside that the two tables could potentially get incredibly large.
2. Have one table of podcasts channels and generate a table for each channel to hold its episodes. The benefits of doing this is that each episode table is separate which provides a performance boost on write heavy tasks since each table can be updated independently. The downside is that the complex structure could make it difficult to maintain.

The expected behaviour is that clients read data from the database more often than they write to it, so the first option is sufficient. It also has the added benefit of being simple to implement and should the need arise, the data can be split out to conform to the second option.

The schema for the podcasts table is shown in **Table 7**:

Table 7: Podcast Table Schema

Field	Type	Description
podcast_id	Int	Auto-incremented podcast ID to explicitly make each row distinct
itunes_id	Int	iTunes identifier for a podcast. Used to uniquely identify a podcast found using the iTunes search API
rss_feed	String	URL pointing to the RSS feed. Used to uniquely identify a podcast and generate an id if the podcast does not have an associated iTunes ID
title	String	Podcast title
author	String	Podcast author(s)
description	String	Podcast description
avg_rating	Double	Average rating of this podcast based on the average ratings of all its episodes

Table 8 shows the schema for the episode table which needs to be more complex as this is where ratings and ad data is stored:

Table 8: Episode Table Schema

Field	Type	Description
episode_id	Int	Auto-incremented episode ID to explicitly make each row distinct
title	String	Episode title
description	String	Description of the episode
pub_date	Timestamp	Publish date of the episode
length	Time	Episode length
avg_rating	Double	Average rating that ByteCast users have given to this episode
new_user_events	JSON	The event collection phase appends new user events that need to be processed to this column
ad_range_model	JSON	The ad time inference phase stores the ad range model in this column. This model is stored so that it can be periodically updated when new user events come in
ad_ranges	JSON	The ad time inference phase stores the resulting ad ranges in this column. This data is what is returned to the client

The schema for the users table is shown in **Table 9**:

Table 9: Users Table Schema

Field	Type	Description
user_id	Int	Auto-incremented user ID to uniquely identify each user
email	String	Unique email address associated with each user, used for sign on
first_name	String	First name of the user
last_name	String	Last name of the user
recommendations	Array	Most recent recommendations for the user

recommendations_timestamp	String	Time of last update for the recommendations
subscriptions	JSON	Podcasts that the user has subscribed to
ratings	JSON	Ratings that the user has provided, used for recommendations

The schemas are relatively simple and are based off the metadata that podcasts come with. Notably, there is no need to store encrypted user credentials since user authentication and authorization is handled by OAuth. To secure the database against external access, IP whitelisting is being employed so that only the EC2 instance has database access by default. Home IP addresses are selectively whitelisted for development purposes.

3.3.4 API Server Technology

The API server has two main tasks: handling requests from clients and running the ad time prediction algorithm that is discussed in **Section 3.3.6**. Client requests mainly deal with getting or setting information in the database. Multiple frameworks can be used for this role but the group members for this project have the most experience with Node.js/Express, Python's Flask framework, and Python's Django framework. In order to choose between the frameworks, three performance tests are used to gauge each framework's effectiveness in the role.

All the tests use the open source WRK HTTP benchmarking tool [21] to create 100 concurrent HTTP connections among 10 threads over 20 seconds which has the effect of simulating an average load environment for each framework. The first test examines the scenario where a framework updates the database and it conducted such that every time the framework receives a request, it updates five rows in a PostgreSQL database table. The more requests it can handle per second, the better the framework is performance wise. The second test has each framework return all the rows in the table upon each request to simulate a situation in which there are lots of database reads occurring. The last test simulates a purely computationally intensive task for each framework by having it calculate the tenth fibonacci number recursively on each request. This test is used to gauge how the server performs under a computationally intensive load like it might see when it runs the ad skipping algorithm. **Table 10** summarizes the results of the tests:

Table 10: API Framework Comparison

	Read Speed (requests/s)	**Write Speed (requests/s)**	**Fibonacci Computation (requests/s)**
Node.js/Express	2070.28	1085.37	5275.01
Python + Django	652.08	574.43	652.33
Python + Flask	734.06	490.48	1142.63

The results show that the Node.js/Express server performed exceedingly better than the two Python alternatives on every test, and is therefore the best option.

3.3.5 API Service Design

This section details the REST API design for the backend server. The API endpoint is the URL that the client application connects to in order to access the backend API. The URL can contain path parameters, and in some cases the client passes a JSON object that contains additional information. JSON is a standard text-based serialization format that consists of attribute-value pairs and that is heavily used in REST APIs [22]. To secure the endpoints, all API requests require an OAuth bearer token to prove that the user is authorized to perform the action or access the requested data. **Table 11** details the API design, with the contents of the curly braces inside the API Endpoint column being the path parameters to the endpoint.

Table 11: API Endpoints

API Endpoint	HTTP Method	Description	Return JSON
/users/create	PUT	Creates a new user. Provide a JSON object in the body with fields for email, name, and gender	{ user_id: int }
/users/{user_id}/recommendations	GET	Get the recommendations for the given user	{ podcast_id0: int, podcast_id1: int, podcast_id2: int, podcast_id3:int, podcast_id4: int }
/users/{user_id}/subscribe/{podcast_id}	PUT	Subscribe the user to the specified podcast	Nothing
/users/{user_id}/unsubscribe/{podcast_id}	PUT	Unsubscribe the user from the specified podcast	Nothing
/users/{user_id}/rate/{podcast_id}	PUT	Give a rating to the specified podcast using the following JSON object: { rating: int }	Nothing
/users/{user_id}/watch/{podcast_id}	PUT	Mark a the specified podcast as watched by the user	Nothing
/users/{user_id}/settings	PUT	Change the settings listed in the JSON object for the given user: { setting1: value, setting2: value }	Nothing
/users/{user_id}/delete	DELETE	Delete the account associated with the user	Nothing

/podcasts/get_id	GET	Get the podcast_id associated with this podcast using the iTunes ID **or** the RSS feed given in the following JSON object: { itunes_id: int, rss_feed: string }	{ podcast_id: int }
/podcasts/{podcast_id}/episodes/get_id	GET	Get the episode_id associated with this episode using the following JSON object: { title: string }	{ episode_id: int }
/podcasts/{podcast_id}/avg_rating	GET	Get the average rating for this podcast	{ rating: double }
/podcasts/{podcast_id}/{episode_id}/avg_rating	GET	Get the average rating for the episode	{ rating: double }
/podcasts/{podcast_id}/{episode_id}/ad_ranges	GET	Get the advertisement ranges for this podcast	{ lastUpdate: long, ranges: [{startTimeMS: double, endTimeMS: double },...] }
/podcasts/{podcast_id}/{episode_id}/events	POST	Post new user events. Provide a JSON object containing an array of event objects, where each event has the form: { start: double, end: double, user: int, type: eventType }	Nothing
/podcasts/{podcast_id}/{episode_id}/sound_byte	PUT	Create a new SoundByte using the title and range given in the JSON object. Resulting URL can be shared on social media and lives for at least one week { title: string, range: { start: long, end: long }	{ url: string }

3.3.6 Ad Time Prediction Design

This component of the backend server system is responsible for capturing user actions sent to the server and efficiently making ad time range inferences based on received user actions and communicate the time ranges back to the iOS client. These two responsibilities can be broken down into two separate phases: the event collection phase and the ad time inference phase.

3.3.6.1 Event Collection Phase

The goal of the event collection phase is to collect user events that are sent to the backend server via a REST call to the events endpoint. User events are simple JSON objects with the following format:

{ start: {startTimeMS}, end: {endTimeMS}, user: {userID}, type: {eventType} }

Where,

- {startTimeMS} is a double-precision floating point number indicating the number of milliseconds since the start of the podcast where this event starts
- {endTimeMS} is a double-precision floating point number indicating the number of milliseconds since the start of the podcast where this event ends
- {userID} is an integer representing the user who sent the actions
- {eventType} is one of the following strings:
    - "MARK", used when the user explicitly marks the locations of advertisements
    - "SKIP_FORWARD" or "SKIP_BACK" for when the user presses the skip forward or skip backwards buttons
    - "FAST_FORWARD" or "REWIND" for when the user presses and holds the fast-forward and rewind buttons for an interval of time

When the server receives these events, it adds them to the new_user_events column of the corresponding episode table. This column contains a JSON array so that the new user events can easily be appended to the existing ones that have not yet been processed.

3.3.6.2 Ad Time Inference Phase

The goal of the ad time inference phase is to take the user events from the event collection phase and try to determine the most likely location of advertisements. The results from this phase are time ranges in the podcast that represent the most likely locations of advertisements. These ranges are given to the iOS client so that it is able to automatically skip over them.

When a user downloads a new podcast on the iOS client, the client first goes through the process of finding out the podcast_id and episode_id using the /podcasts/get_id and /podcasts/{podcast_id}/episodes/get_id APIs. Once it determines this information, it is able to make a request to the /podcasts/{podcast_id}/episodes/{episode_id}/ad_ranges endpoint with the appropriate podcast ID and episode ID. The server responds to this request using the simplified logic in **Figure 12**:

```
void getAdRanges(podcast_id, episode_id) {
  Row row = getDatabaseRow(podcast_id, episode_id);
  if (row != null) {
    JSON entry = row.getColumn("ad_ranges");
    if (entry.lastUpdate.isOlderThanTwoHours() &&
        row.getColumn("new_user_events").notEmpty() { // minimize ad range recalculation
      entry = updateAdRanges(row);
      updateDatabaseRow(entry);
    }
    return entry;
  } else {
    createDatabaseRow(podcast_id, episode_id);
    return { lastUpdate: 0, ranges: [] };
  }
}
```

Figure 12: Pseudocode for responding to a request for ad ranges

The pseudocode tries to minimize the number of times that the ad ranges are recalculated by storing the result of the last computation as well as a timestamp into the ad_ranges column of the table. The ranges are only recalculated when they are more than two hours old and when there are new user events that can be used to update the estimate. The most interesting and technically challenging part of the ad time inference phase is the implementation of the "*updateAdRanges()*" method from the above pseudocode. The purpose of this method is to take the contents of the new_user_events column, integrate it into the existing data model that describes the events that have been received in the past (using the ad_range_model column), and output a new set of improved ad ranges. Some of the ideas used in the construction of this method come from a technique known as Bayesian filtering. Bayesian filtering attempts to compute estimates for the current state of a system given past measurements [23]. It can be used to estimate an unknown probability density function over time using past measurements. Typically, Bayesian filtering is used to estimate real-world continuous functions in applications such as robotics [24]. To simplify the mathematics required to implement this sort of algorithm, the "*updateAdRanges()*" method considers the length of a podcast episode to be divided into discrete 1-second intervals. Each 1 second interval keeps a positive integer count. A count of 0 means that the server has no reason to believe that the time range in question is part of an advertisement. A count of something greater than 0 indicates that there is a certain probability that the time range in question is part of an advertisement. Each time the server receives *SKIP_FORWARD* or *FAST_FORWARD* events, all the time intervals that these events encompass are incremented by 1. When the server receives *SKIP_BACK* or *REWIND* events, and the time ranges for these events overlap with the time ranges that the same user submitted for *SKIP_FORWARD* or *FAST_FORWARD* events, these overlapping time ranges are decremented by 1. The idea here is that most users skip forward or fast forward a little too far past the end of the ad, and then skip back or rewind a little in order to get to the beginning of the non-ad content. Lastly, if the server receives a *MARK* event, then all time intervals that this event encompasses are incremented by 2. Since the user is explicitly marking the location of an ad, more trust is put into these events and thus they are incremented further than the other event types. Once all user events have been accounted for, what is left is a discrete non-normalized probability density function (PDF) that shows the time ranges that are most likely to contain advertisements.

Once the user events have been processed, the last step is to detect the advertisement time ranges. Intuitively, by looking at the PDF one would be able to visually identify the ranges that are most likely to contain advertisements. The challenging part is to algorithmically identify these ranges in an efficient manner; ideally in O(n) time complexity, where n is the number of seconds in the podcast. The solution arrived upon iterates over all of the seconds in the podcast and computes a moving average using the stored counts. This acts as a simple low-pass filter in order to reduce some of the noise in the data and smooth the overall function. During this initial pass, the max value of any time range in the moving average is retained in order to help with the next step. During the next step, a pass is made through the recently computed moving average in order to identify the final ad ranges. In order to compute the ranges, some heuristics are used. These heuristics were arrived upon partly through trial-and-error, and are likely to continue to evolve during future design iterations. In order for a given time range within the moving average to be considered an advertisement, the following three conditions must be met:

1. Time range must have a count value of greater than or equal to 3
2. Time range must have a count value of greater than or equal to half of the max value calculated in the first pass
3. Time range must be at least 10 seconds in length

The first condition ensures that multiple users have signalled that this time range could potentially contain an advertisement. The second condition attempts to filter out false-positives by ignoring ranges that have small count values relative to the max count value found in the first pass. The third condition ignores ranges that are too short to be an ad read. Once the ad ranges have been updated, the new_user_events column is cleared and the model is serialized into the ad_range_model column so that it can be reused the next time that more user events are added. Initial tests of this algorithm have shown that with test data, the algorithm is able to determine the advertisement ranges to within ±5 seconds with data from at least 10 users. As the prototype is further built out and released as a beta, it will be possible to test the algorithm using real-world data and further refine the heuristics used.

3.4 Recommendations Subsystem Design

The recommendations subsystem is comprised of five major components. These components are the Remote Procedure Call (RPC) interface, Database interface, Events manager, Item Recommender and Item Scorer. The recommendation framework used is Lenskit which provides the necessary tools and interfaces to build a complete recommender system. The framework requires the use of the Java programming language. Alternatives such as PredictionIO and Mortar were considered before choosing the framework in use.

Three frameworks were considered for the design of the recommendation framework before finally selecting Lenskit. The three frameworks were Lenskit, PredictionIO and Mortar. As shown in **Table 12,** the decisions were based primarily on cost and then on other attributes such as licensing and scalability. The alternatives were eventually rejected since PredictionIO provides a recommender framework that requires a Hadoop cluster with Hadoop Distributed File System (HDFS) that cannot be locally run on the same servers as the rest of the backend system. Mortar is heavily dependent on Amazon Web Services (AWS) for computing the recommendations and Amazon S3 to use as persistent storage. This does not

work for this project because the use of Amazon Web Services for scalable infrastructure costs a significant amount of money.

Table 12: Decision matrix showing the three metrics that drive the framework decision

Alternative	Cost (50%)	Open Source (25%)	Scalability (25%)	Total
Lenskit	0.5	0.25	0.1	0.85
PredictionIO	0.3	0.25	0.25	0.8
Mortar	0.3	0	0.25	0.55

The database interface is used to store the results of computation in persistent memory. However, the database runs on a different process, so a special method of communication is required. A class is designed that handles serializing and deserializing all data before it is sent to the database. To make the actual connection to the database, the Java Database Connectivity (JDBC) module is used. This module uses a driver which allows Java programs to connect to a PostgreSQL database using standard, database independent Java code. JDBC connectors are universally used in industry applications where a connection from a Java application to a database is required.

The RPC interface is used by the main backend server process to communicate events across the network to the recommender process and make recommendation requests for a user on behalf of a client. Frameworks considered for making such calls were Apache Thrift and Google RPC (gRPC). gRPC is geared towards making RPC calls from mobile applications whereas Apache Thrift is more general purpose. Additionally, the team has some experience with Apache Thrift due to prior coursework. Due to the reason stated, Apache Thrift is picked as the framework of choice for inter process communication between modules.

The Lenskit framework builds a model before calculating the recommendations. This model can be serialized and stored in persistent storage. Every time a recommendation is requested, the model built last is used to calculated the recommendations for the requested user. Every user is guaranteed to have three fresh recommendations calculated at least once a day using the latest available model at that time. Since each recommendation may take up to 25 seconds to generate, recommendations are calculated for multiple users at the same time by taking advantage of parallel processing using threads. This way all instances of the engine may share the same model while computing recommendations for multiple users at the same time.

The event manager is a class that handles the incoming events from the client that are forwarded through the backend server. These events come in through the RPC interface discussed above. Two major events are currently defined that are used to make recommendations. The generic data format is designed to be extensible if more events are deemed necessary in the future. This format looks like the following,

Event data format: `[ EVENT_TYPE, [EVENT_DATA] ]`

Table 13 below shows the two events and the associated format that the backend server is expected to provide. The events received are reformatted and stored in the PostgreSQL database on the backend server using SQL calls made through the database interface.

Table 13: Data format of different events that the recommender expects to receive.

Event	Format	Description
Listened	`[EVT_FIRST, [podcast_guid, episode_guid]]`	User has listened to a podcast.
Rate	`[EVT_RATE, [podcast_guid, episode_guid, rating_score]]`	User has rated a podcast.

The item recommender and scorer are important parts of the recommendation system. A podcast is considered to be an item. The item scorer finds the similarity (cosine similarity) between podcasts and recommends the top 5 podcasts that are most similar to a certain podcast [25]. It only ranks items that are immediate neighbours of the original podcast. This neighbourhood can be extended if five recommendations cannot be found. The "goodness" of these recommendations is difficult to assess and the best method to do this is to gather user feedback or perform A/B testing. However, this is not possible to do until there exists an application in active production.

Initializing the engine and building the similarity model to compute recommendations takes approximately 25 seconds. This time accounts for starting the recommendation engine from a cold start where the similarity model is rebuilt from scratch for every set of recommendations generated. The entire application must support up to 5000 users which means the recommendation engine must be able to compute recommendations for all those users at least once a day. This adds up to approximately 34 hours of computation on a single thread. However, computing recommendations can be heavily parallelized and multithreading with even just two threads provides a significant performance optimization. On a server with two cores running two threads, the computation time is reduced by half to approximately 18 hours. For the purposes of this project, the recommender engine spawns up to 8 threads for computing the recommendations so the total time for 5000 users should be well under 24 hours. This allows the platform to guarantee that each user gets fresh recommendations every day.

4 Discussion and Project Timeline

4.1 Evaluation of Final Design

The objective of this project is to design a podcasting app that improves the consumption appeal of podcasts for users by including features such as advertisement-skipping, smart recommendations and improved shareability of podcasts. The proposed design in this report meets this objective by

implementing client-side iOS and web applications along with a server-side API endpoint, database and recommendation engine. Overall, the design meets all of the original design specifications in a strictly theoretical sense.

4.2 Use of Advanced Knowledge

This project uses upper-year ECE knowledge from five main subject areas: ECE 356 Databases, ECE 358 Networking, ECE 452 Software Architecture, ECE 316 Probability and ECE 454 Distributed Systems. Databases are used primarily in the backend server subsystem as well as the iOS subsystem; the backend subsystem stores user accounts, podcast metadata as well as user recommendations and the iOS subsystem stores podcast metadata for locally stored podcasts as well as user interaction statistics. The iOS subsystem, backend server subsystem and the web subsystem all use some form of networking knowledge. All of these subsystems are able to send and receive messages through HTTP requests that are sent over the network. In the case of the recommendations subsystem, communication with the backend is handled through the use of RPCs which require knowledge from distributed systems and networking courses. Numerous software architecture patterns such as delegation, singletons and a layered architecture based on functionality are incorporated into the iOS subsystem. The Database subsystem applies probability concepts such as Bayes theorem in order to make a model to determine accurate times for ad skipping.

4.3 Creativity, Novelty and Elegance

This project is novel because currently, there aren't any podcast applications which provide features such as ad-skipping and recommendations in a single application. To the best of our knowledge, this application is also the first to ever attempt to skip content-embedded advertisements through the use of crowdsourced data. The current application uses a multitude of technologies to implement the different subsystems and integrates them to create a creative and elegant solution to the given problem.

One example of elegance can be seen in the iOS subsystem which takes advantage of a new paradigm introduced in Swift called Protocol Oriented Programming (POP). Protocol Oriented Programming is the concept of using protocols rather than defining classes and using inheritance to solve common programming problems. Another example of elegance can be seen in the generation of recommendations which makes use of parallel processing to compute recommendations for multiple users at the same time.

4.4 Student Hours

Table 14 shows how many hours each student has worked so far on the project.

Table 14: Student Hours

Group Member	# of Hours
[redacted]	43
[redacted]	50
[redacted]	50
[redacted]	83
[redacted]	48

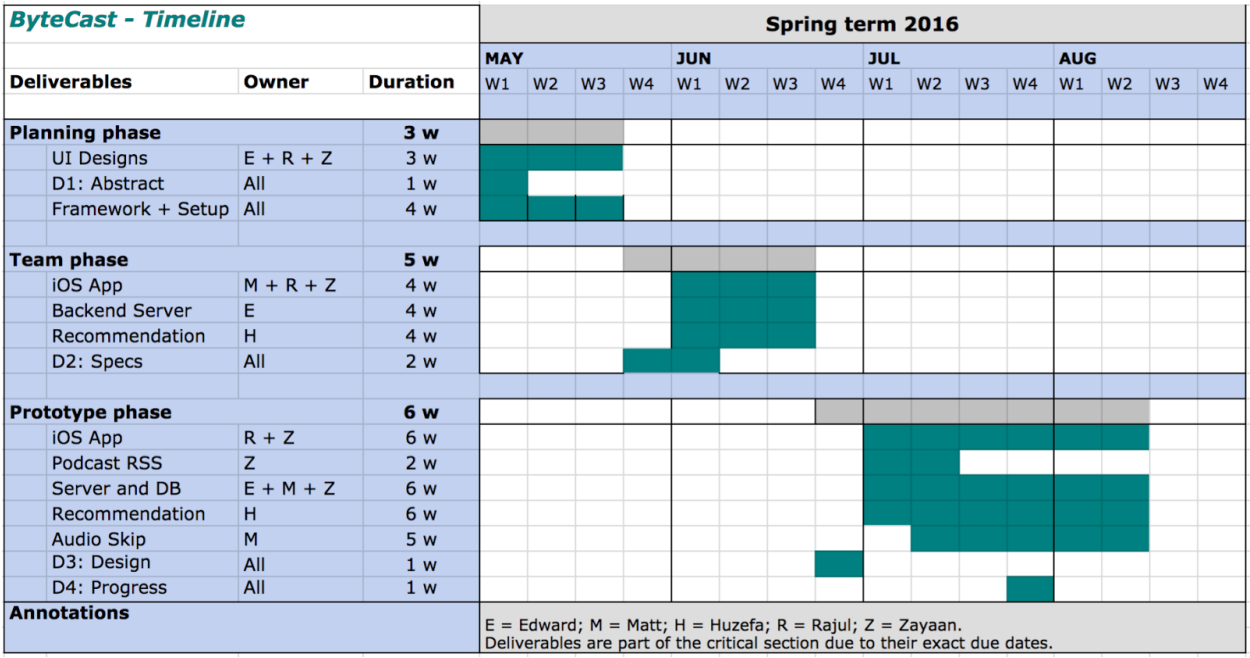
4.5 Potential Safety Hazards

Since this is a pure software project, there are no real physical safety hazards associated with the design and construction of the project.

4.6 Project Timeline

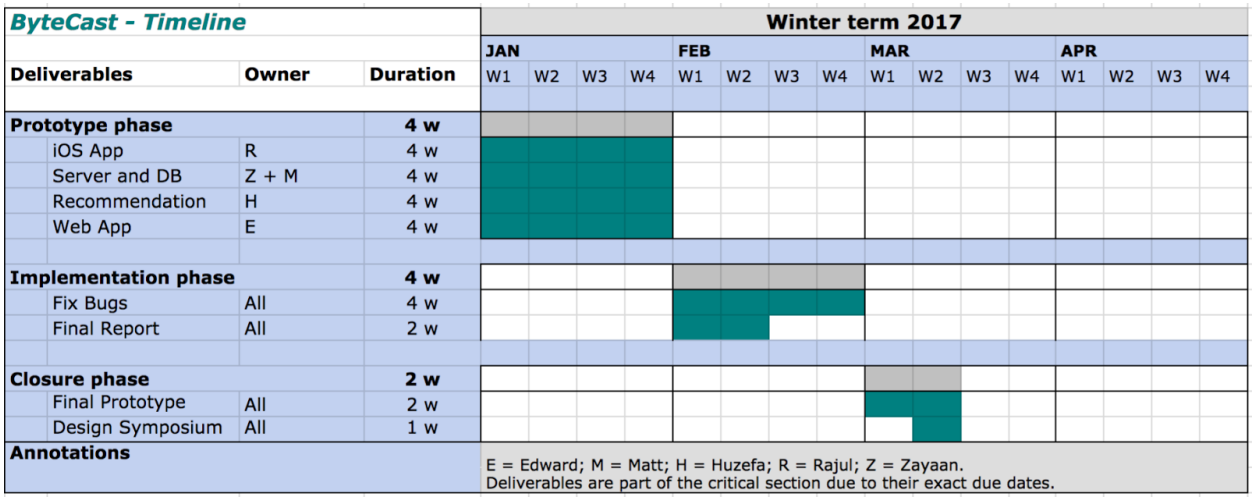
Figure 13 and **Figure 14** show the proposed project timelines from now until the end of the Winter 2017 term.

Figure 13: Spring 2014 Project Timeline



ByteCast - Timeline			Spring term 2016															
			MAY				JUN				JUL				AUG			
Deliverables	Owner	Duration	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4
Planning phase		**3 w**																
UI Designs	E + R + Z	3 w																
D1: Abstract	All	1 w																
Framework + Setup	All	4 w																
Team phase		**5 w**																
iOS App	M + R + Z	4 w																
Backend Server	E	4 w																
Recommendation	H	4 w																
D2: Specs	All	2 w																
Prototype phase		**6 w**																
iOS App	R + Z	6 w																
Podcast RSS	Z	2 w																
Server and DB	E + M + Z	6 w																
Recommendation	H	6 w																
Audio Skip	M	5 w																
D3: Design	All	1 w																
D4: Progress	All	1 w																
Annotations			E = Edward; M = Matt; H = Huzefa; R = Rajul; Z = Zayaan. Deliverables are part of the critical section due to their exact due dates.															

Figure 14: Winter 2017 Project Timeline



ByteCast - Timeline			Winter term 2017															
			JAN				FEB				MAR				APR			
Deliverables	Owner	Duration	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4
Prototype phase		**4 w**																
iOS App	R	4 w																
Server and DB	Z + M	4 w																
Recommendation	H	4 w																
Web App	E	4 w																
Implementation phase		**4 w**																
Fix Bugs	All	4 w																
Final Report	All	2 w																
Closure phase		**2 w**																
Final Prototype	All	2 w																
Design Symposium	All	1 w																
Annotations			E = Edward; M = Matt; H = Huzefa; R = Rajul; Z = Zayaan. Deliverables are part of the critical section due to their exact due dates.															

References

[1] N. Vogt, "Podcasting: Fact Sheet", *Pew Research Center's Journalism Project*, 2015. [Online]. Available: http://www.journalism.org/2015/04/29/podcasting-fact-sheet/. [Accessed: 29- May- 2016].

[2] J. Morgan, "How Podcasts Have Changed in Ten Years: By the Numbers", *Medium*, 2015. [Online]. Available: https://medium.com/@slowerdawn/how-podcasts-have-changed-in-ten-years-by-the-numbers-720a6e984e4e#.366lbn43z. [Accessed: 29- May- 2016].

[3] M. Massé, *REST API Design Rulebook*. Sebastopol, CA: O'Reilly, 2012.

[4] D. Marshall, "Remote Procedure Calls (RPC)", Cardiff School of Computer Science & Informatics, 1999. [Online]. Available: https://www.cs.cf.ac.uk/Dave/C/node33.html. [Accessed: 29- May- 2016].

[5] M. Ekstrand, "Tools and Experiments for Identifying Recommender Differences", Ph.D, University of Minnesota, 2014.

[6] "Delegation", developer.apple.com, 2016. [Online]. Available: https://developer.apple.com/library/ios/documentation/General/Conceptual/DevPedia-CocoaCore/Delegation.html. [Accessed: 29- Jun- 2016].

[7] “ccgus/fmdb”, Github, 2016. [Online]. Available: https://github.com/ccgus/fmdb. [Accessed: 28-Jun-2016].

[8] “Realm”, Realm, 2016 [Online]. Available: https://realm.io. [Accessed: 28-Jun-2016].

[9] “Introducing Realm | Realm.io”, Realm, 2016 [Online]. Available: https://realm.io/news/introducing-realm/. [Accessed: 28-Jun-2016].

[10] “Audio Toolbox”, developer.apple.com, 2016. [Online]. Available: https://developer.apple.com/library/tvos/documentation/MusicAudio/Reference/CAAudioTooboxRef/index.html. [Accessed: 28-Jun-2016].

[11] “Core Audio Overview”, developer.apple.com, 2016. [Online]. Available: https://developer.apple.com/library/mac/documentation/MusicAudio/Conceptual/CoreAudioOverview/Introduction/Introduction.html. [Accessed: 28-Jun-2016].

[12] “delannoyk/AudioPlayer”, Github, 2016. [Online] Available: https://github.com/delannoyk/AudioPlayer. [Accessed: 28-Jun-2016].

[13] “Singleton”, developer.apple.com, 2016. [Online]. Available: https://developer.apple.com/library/ios/documentation/General/Conceptual/DevPedia-CocoaCore/Singleton.html. [Accessed 28-Jun-2016].

[14] “UIKit Framework Reference”, developer.apple.com, 2016. [Online]. Available: https://developer.apple.com/library/ios/documentation/UIKit/Reference/UIKit_Framework/. [Accessed: 28-Jun-2016].

[15] “UITabBarController Framework Reference”, developer.apple.com, 2016. [Online]. Available: https://developer.apple.com/library/ios/documentation/UIKit/Reference/UITabBarController_Class/. [Accessed: 28-Jun-2016].

[16] “Generics”, developer.apple.com, 2016. [Online]. Available: https://developer.apple.com/library/ios/documentation/Swift/Conceptual/Swift_Programming_Language/Generics.html. [Accessed: 28-Jun-2016].

[17] "Free Cloud Services – AWS Free Tier", Amazon Web Services, Inc., 2016. [Online]. Available: https://aws.amazon.com/free/. [Accessed: 29- Jun- 2016].

[18] "Pricing | Heroku", *Heroku.com*, 2016. [Online]. Available: https://www.heroku.com/pricing. [Accessed: 29- Jun- 2016].

[19] "Heroku Postgres - Add-ons - Heroku Elements", *heroku.com*, 2016. [Online]. Available: https://elements.heroku.com/addons/heroku-postgresql. [Accessed: 29- Jun- 2016].

[20] "Sign Up For Your Free Trial", Google Developers, 2016. [Online]. Available: https://cloud.google.com/free-trial/. [Accessed: 29- Jun- 2016].

[21] "wg/wrk", *GitHub*, 2016. [Online]. Available: https://github.com/wg/wrk. [Accessed: 28- Jun- 2016].

[22] "JSON", Wikipedia, 2016. [Online]. Available: https://en.wikipedia.org/wiki/JSON. [Accessed: 20- Jun- 2016].

[23] S. Särkkä, Bayesian filtering and smoothing. Cambridge, U.K.: Cambridge University Press, 2013.

[24] "Recursive Bayesian estimation", Wikipedia, 2016. [Online]. Available: https://en.wikipedia.org/wiki/Recursive_Bayesian_estimation. [Accessed: 20- Jun- 2016].

[25] "Recommendation Engine Basics | Mortar Help & Tutorials", help.mortardata.com, 2016. [Online]. Available: http://help.mortardata.com/data_apps/recommendation_engine/recommendation_engine_basics#toc_1ItemItemAffinity. [Accessed: 27- Jun- 2016].